VHDL Implementation of NIST Ascon AEAD128 on a Nexys A7 FPGA

Kroix Jones
Electrical Engineering Undergraduate Researcher
Howard University
Research Advisor: Professor Wenjie Che

Abstract

This paper describes a VHDL-2008 implementation of the standardized NIST Ascon-AEAD128 algorithm for a Nexys A7 100T FPGA. The design uses one shared 320-bit permutation datapath and completes one Ascon round per clock cycle. A 128-bit streaming interface accepts associated data and payload bytes, including empty and partial blocks. Encryption returns ciphertext and a tag. Decryption returns tentative plaintext and releases it only after tag verification. The portable test flow passed all 1,089 official known-answer vectors in both directions with GHDL and Vivado XSim. Generated Verilog was also checked with Icarus Verilog and Verilator. Vivado 2023.2 completed synthesis, placement, routing, timing analysis, and bitstream generation. Vivado used 1717 slice LUTs and 1372 slice registers, with no block RAM. The routed design met the 100 MHz constraint with 2.941 ns worst setup slack. Vivado estimated 0.137 W total on-chip power and generated the FPGA bitstream. Physical board programming and demonstration testing remain pending. An application experiment fine-tuned GPT-2 for 40 updates in Ubuntu under WSL 2 and exported a 497,904,640-byte FP32 checkpoint. A 1,024-byte weight sample was encrypted and authenticated by the same RTL core in XSim in 801 cycles, or 8.01 microseconds at a simulated 100 MHz clock. Decryption recovered the original sample exactly. This result is behavioral simulation latency rather than a physical board measurement.

I. Research objective

The goal of this work is to build an understandable FPGA implementation of authenticated encryption. The core must follow the final standard, handle normal streaming stalls, and make authentication failure visible to the surrounding system. The first version favors a clear control path over maximum throughput. It is meant to support later research on faster datapaths and hardware security countermeasures without changing the external transaction format.

II. Ascon-AEAD128 background

NIST SP 800-232 defines Ascon-AEAD128 with a 128-bit key, 128-bit nonce, 128-bit authentication tag, 128-bit data rate, and a 320-bit internal state [1]. The state is divided into five 64-bit words. Initialization and finalization use a 12-round permutation. Associated-data and full payload blocks use an 8-round permutation. Each round applies a constant to one state word, evaluates 64 five-input S-box slices in parallel, and then performs word-specific rotations and XOR operations.

The final NIST version was selected instead of legacy Ascon-128 or Ascon-128a. The final standard changed the initialization value and changed byte mapping to little endian [1]. For this core, byte lane 0 is bits 7 through 0 of the 128-bit port. Padding places the byte value 01 immediately after the last valid byte. Using the official mapping avoids a common result where a design is internally consistent but does not match current NIST vectors.

Initialize	Absorb AD	Domain	Process data	Finalize	**Tag**
p12	p8	separate	p8	p12	**128 bits**

Fig. 1. Ascon-AEAD128 processing phases, adapted from NIST SP 800-232 [1].

III. Hardware architecture

The datapath contains five 64-bit state registers and one combinational round. A small round counter selects the required constants and returns the updated state after 8 or 12 clocks. Reusing one round reduces duplicated logic and keeps the relationship between the VHDL and the algorithm visible. The same engine serves initialization, associated data, payload processing, and finalization.

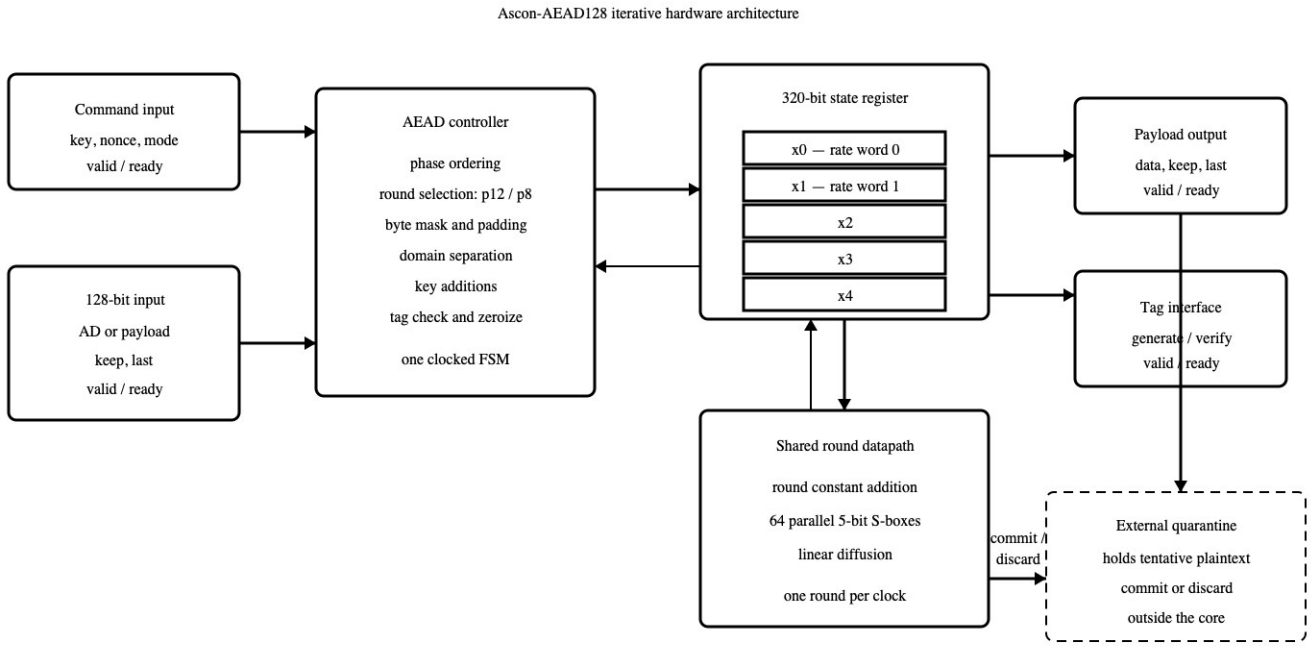


Fig. 2. Original block diagram of the implemented architecture.

The controller accepts a command containing the key, nonce, and direction. Associated data must finish before payload begins. Each stream beat includes 16 keep bits and a last flag. Keep bits must start at lane 0 and remain contiguous. An empty phase uses a terminal beat with no keep bits set. Any invalid ordering or mask produces a protocol error and sends all transaction storage through the same zeroization state used after a normal result.

IV. Authentication-safe decryption

A decryption core cannot know whether plaintext is authentic until it processes the final tag. Storing every possible message inside the cryptographic core would make its memory size depend on the application. This design sends tentative plaintext to an external quarantine buffer instead. A correct tag pulses commit. A wrong tag pulses discard and never pulses commit. The interface therefore makes the safe release rule explicit, but the system integrator must enforce it.

V. Controller and verification

The finite-state machine was made for this streaming architecture. Start states issue a one-cycle request to the permutation engine. Wait states prevent another request until the current permutation ends. Output states hold data stable during backpressure. Tag states either wait for the encryption tag to be accepted or wait for a decryption tag to be supplied.

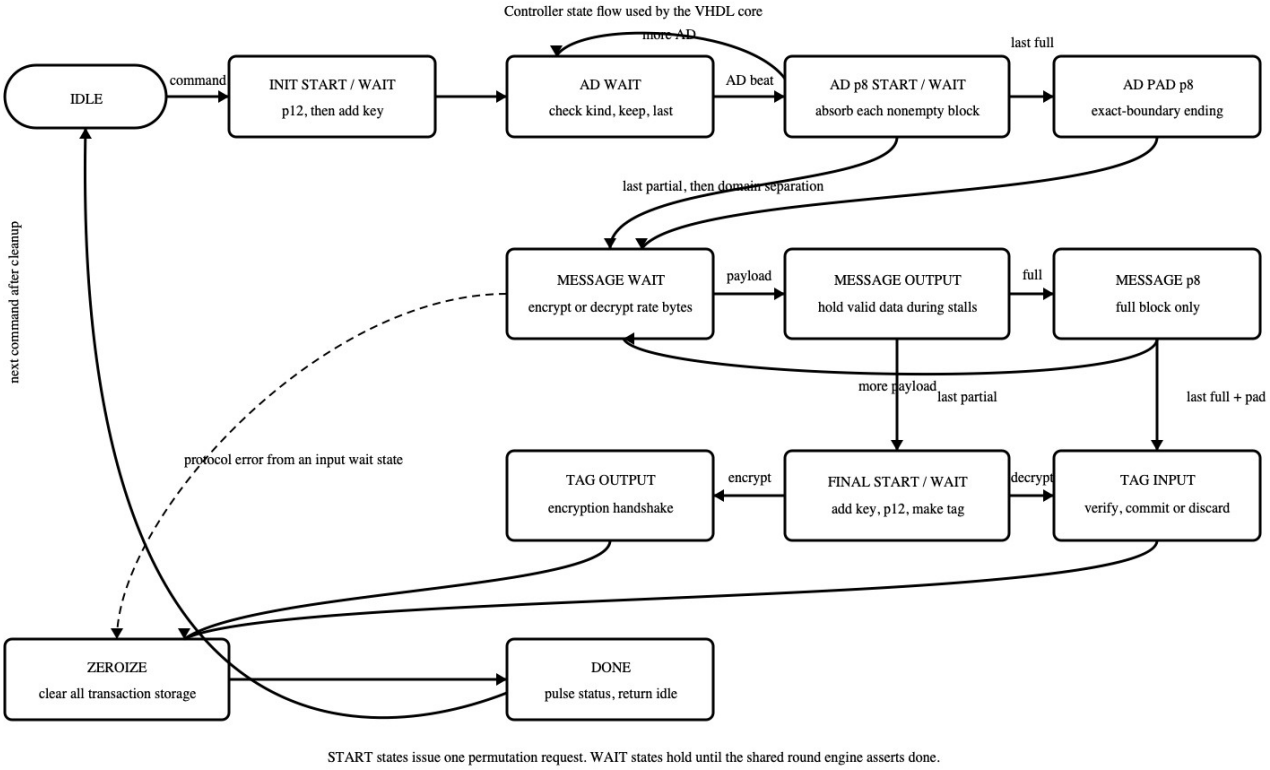


Fig. 3. Original controller state flow used by the VHDL core.

Verification began with an independent Python model. The model was compared with the official ascon-c known-answer file pinned to a recorded source revision [2]. Cocotb then drove the same vectors through the VHDL core. The full run covered all 1,089 combinations of associated-data and plaintext lengths from 0 through 32 bytes, with encryption followed by decryption. Separate tests stalled each output, reset an active transaction, sent invalid masks and phase ordering, checked fixed-seed messages up to 80 bytes, and changed the key, nonce, associated data, ciphertext, and tag.

Verification level	Cases	Result	Evidence
Python reference	81 boundary pairs	PASS	official KAT comparison
GHDL direct VHDL	1,089 × 2	PASS	encrypt and decrypt
Protocol/security	9 test groups	PASS	stalls, reset, zeroization
GHDL TextIO	1,089 × 2	PASS	two output text files
Vivado XSim	1,089 × 2	PASS	official KAT TextIO bench
Icarus netlist	1,089 × 2	PASS	GHDL-generated Verilog
Verilator netlist	1,089 × 2	PASS	GHDL-generated Verilog
Vivado implementation	placed and routed	PASS	reports and bitstream
Nexys A7 board	four demo vectors	PENDING	programming and hardware test

Table I. Verification and implementation status as of September 18, 2026.

VI. Vivado flow and FPGA demonstration

The repository contains Tcl scripts for Vivado 2023.2 and an XDC file for the Nexys A7 100T part xc7a100tcsg324-1. The constraint sets the board clock to 100 MHz. A pure VHDL XSim testbench writes simulation_results.txt for human review and simulation_vectors.txt for scripts. The same file-driven bench was checked with GHDL across all 1,089 vectors in both directions. XSim used the identical TextIO input and completed 2,178 encryption and decryption checks with no failures. The official-vector TextIO run measured 34 to 88 command-to-done cycles. For a 32-byte payload, the mean across all 33 associated-data lengths was 72.636 cycles, equal to 352.441 Mb/s at 100 MHz. These are cycle-based simulation results, not a measured maximum clock rate. The implementation results were extracted from the routed Vivado reports. Vivado used 1717 slice LUTs and 1372 slice registers, with no block RAM. The routed design met the 100 MHz constraint with 2.941 ns worst setup slack. Vivado estimated 0.137 W total on-chip power and generated the FPGA bitstream. Physical board programming and demonstration testing remain pending. The original routed report included a CFGBVS-1 configuration warning. The repository XDC now sets CFGBVS to VCCO and CONFIG_VOLTAGE to 3.3 for the Nexys A7 configuration bank. Physical-board validation remains pending. The power value is a Vivado estimate rather than a board measurement.

The board wrapper stores four official vectors: empty input, a one-byte partial block, one full block, and a 17-byte two-block case. Switches choose the vector, encryption or decryption, payload or tag display, block number, and byte number. The center button starts a run and the upper button resets the design. LEDs display the selected result byte, busy and done state, authentication pass or fail, and a protocol error.

VII. WSL Ubuntu environment

An application experiment completed on October 6, 2026 used trained GPT-2 weight bytes as the input to the existing Ascon-AEAD128 VHDL core. The workflow placed GPT-2 training in Ubuntu under Windows Subsystem for Linux 2 (WSL 2) and retained Vivado 2023.2 on Windows for RTL simulation. This allowed the Linux training process to use the laptop NVIDIA GPU without a separate desktop virtual machine or a second Vivado installation [7], [8].

Component	Recorded configuration
Host	Windows 11 Home; GIGABYTE AORUS 15P YD
CPU and memory	Intel Core i7-11800H; 8 cores, 16 logical processors; 32 GB RAM
GPU	NVIDIA GeForce RTX 3080 Laptop GPU; 8,192 MiB VRAM
Linux environment	Ubuntu 22.04.5 LTS under WSL 2; user kroix
Python and PyTorch	Python 3.10; PyTorch 2.11.0+cu128
Other libraries	Transformers 5.18.0; NumPy 1.26.4
GPU driver	Windows NVIDIA driver 576.52; nvidia-smi reports CUDA capability 12.9
Hardware tool	Vivado 2023.2 and XSim on Windows

Table II. Software and hardware environment recorded for the GPT-2 experiment.

Ubuntu was installed from an administrator PowerShell session with wsl --install -d Ubuntu-22.04. The distribution was confirmed as WSL version 2, and nvidia-smi inside Ubuntu detected the RTX 3080. WSL uses the Windows NVIDIA driver; a separate Linux NVIDIA display driver was not installed [8]. The cu128 suffix identifies the PyTorch CUDA 12.8 build. The CUDA version printed by nvidia-smi describes driver capability and does not by itself establish an installed CUDA compiler.

The project was cloned into ~/projects/SAGE_Accelerator/llmc-gpt2, and a Python virtual environment named .venv kept its dependencies separate. Git, build-essential, python3-pip, and python3-venv supplied the basic development tools. GPU access was verified by allocating a 256 by 256 tensor on CUDA, multiplying it, and synchronizing the device. This calculation passed before training began. The native C/CUDA trainer was not used for GPU training; the Makefile reported that nvcc, MPI, and NCCL development support were unavailable. These components were unnecessary for the PyTorch run and CPU correctness test.

Appendix A records the shell commands and final export code. The setup and measurements below describe this specific small experiment; they do not establish production model quality or physical FPGA performance.

VIII. GPT-2 training and weight export

The software was taken from SPIRE-GMU/SAGE_Accelerator, llmc-gpt2, at commit 4e586d80b61ad7577c64391202a8c43104728f1d [9]. This folder contains an llm.c implementation and the PyTorch reference trainer train_gpt2.py. The pretrained gpt2 model was loaded through Hugging Face. The run fine-tuned existing GPT-2 weights for 40 optimizer updates; it did not train GPT-2 from random initialization.

The Tiny Shakespeare preparation script downloaded approximately 1.06 MiB of text and generated two token files. The training split contained 305,260 tokens in 611,544 bytes, and the validation split contained 32,768 tokens in 66,560 bytes. The binary sizes include a 1,024-byte dataset header. Training used the training file and validation used the separate validation file.

Before fine-tuning, a zero-update run with batch size 4 and sequence length 64 generated the pretrained FP32 and BF16 checkpoints, tokenizer, and PyTorch debug state. The CPU test was built with make test_gpt2 and executed with OMP_NUM_THREADS=8. It passed the logits, initial loss, gradient comparisons, and all ten expected training-loss checks, ending with overall okay: 1. An earlier batch-size-1 debug file produced correct tensor comparisons but failed the fixed training-loss expectations; regenerating the reference with the standard batch size 4 resolved the mismatch. The zero-update timing warning nanms only reflected an empty timing list.

Training setting	Value
Compute and precision	One RTX 3080; FP32; compilation disabled
Microbatch and sequence	1 sequence of 64 tokens
Effective batch	256 tokens; 4 gradient accumulation steps
Optimizer updates	40; overfit_single_batch disabled
Learning rate	0.0001; no warmup; default decay fraction 1.0
Validation sampling	Every 10 updates and at completion; 5 batches per check
Output directory	runs/ascon_demo_40

Table III. Settings for the GPU fine-tuning demonstration.

Validation loss was 5.404110 before training and 3.810759 after 40 updates. Intermediate checks after 10, 20, and 30 updates were 4.078157, 3.942588, and 3.860151. Each value averages five validation batches rather than the entire validation split. The trainer reported a mean of 132.058 ms over its final 19 recorded iteration timings and peak PyTorch allocated GPU memory of 2,416 MiB. These iteration timings are not the total elapsed run time and include any validation work scheduled in a timed iteration.

The supplied PyTorch bridge exports initialization weights before its training loop. An additional export block was therefore added after the loop to call write_model(raw_model, ..., dtype="float32") and save the updated weights as gpt2_trained_fp32.bin. The file size was 497,904,640 bytes: a 1,024-byte header plus 497,903,616 bytes of FP32 parameter data, including vocabulary padding from 50,257 to 50,304 entries. This is approximately 474.84 MiB. The serialized payload contains 124,475,904 FP32 slots; the unpadded model has 124,439,808 unique parameters.

A 1 KiB sample was extracted by skipping the checkpoint header and reading the next 1,024 bytes, equivalent to 256 FP32 values from the first token embedding. The file was verified against that trained payload, and the trained checkpoint was confirmed to differ from the pretrained checkpoint. The sample was copied to Windows as gpt2_trained_weights_1KiB.bin. It represents only a small portion of model weights and cannot reconstruct the model on its own.

IX. Ascon encryption of trained weight bytes

The original official-vector TextIO testbench stores only two 16-byte payload blocks, so it cannot accept the entire 1 KiB sample. A separate runner, verification/gpt2_weights_demo.py, generated a dedicated tb_gpt2_weights testbench and an isolated Vivado project under build/gpt2_weights_demo. The runner reused the existing ascon_pkg.vhd, ascon_permutation.vhd, and ascon_aead128_core.vhd. The application test streamed all 64 full 16-byte beats as one AEAD message; the original RTL and official-vector testbench were not altered.

The Python Ascon reference first passed all 1,089 pinned official known-answer vectors. It then computed the expected ciphertext and tag for the actual trained-weight sample. The testbench checked every ciphertext block and the tag, then decrypted the ciphertext and checked all recovered bytes and successful authentication. Full block dumps were preserved in full_output.txt, while simulation_results.txt and simulation_vectors.txt recorded the PASS results and cycle counts. The runner verified the complete byte arrays before producing report.json.

Measurement or input	Recorded result
Plaintext and associated data	1,024 weight bytes; 0 associated-data bytes
Demo key	000102030405060708090a0b0c0d0e0f
Demo nonce	101112131415161718191a1b1c1d1e1f
Clock	100 MHz; 10 ns per cycle
Encryption	801 command-to-done cycles; 8.01 microseconds; PASS
Decryption	801 command-to-done cycles; 8.01 microseconds; PASS
Payload throughput	1,022.72 Mb/s calculated from simulated latency
Authentication tag	e3a4ec0147a1f6681ba8f8aeb188a49f
Ciphertext and tag sizes	1,024-byte ciphertext and a separate 16-byte tag
Round-trip integrity	Recovered 1,024 bytes exactly match the input sample

Table IV. Behavioral XSim results for one 1 KiB trained-weight sample.

The cycle counter was sampled when a ready command was offered and again when done was observed. Thus the interval includes command submission, initialization, the empty associated-data phase, all payload handshakes, finalization, tag handling, and completion. The driver accepted each output beat before submitting the next input beat. The measurement uses that testbench schedule and must not be described as an ideal unstalled datapath limit. The latency calculation is 801 cycles times 10 ns per cycle, giving 8,010 ns or 8.01 microseconds. Payload throughput is 1,024 times 8 bits divided by 8.01 microseconds.

The source sample SHA-256 is recorded below. The Windows copy matched the Ubuntu export, binding the encryption result to the actual trained-weight bytes. The demo key and nonce are public test inputs. A deployed encryption system would require protected keys and unique nonces for each message under a given key; repeating this fixed pair is appropriate only for reproducing this test.

```
fa00e0b873c026e9c25298fc1428872a0968
0e11e0ec5ac74af29fcab3e34541
```

The 8.01-microsecond result is functional RTL simulation at a chosen 100 MHz clock. No physical board encryption, UART/USB transfer, DMA, PC file I/O, or post-route application timing was measured. The small-payload result of 352.441 Mb/s in Section VI averages different associated-data lengths, whereas this case has empty associated data and a much larger payload. The numbers therefore describe different workloads and should not be treated as a controlled architectural speedup.

X. Reducing latency at a fixed clock frequency

The 1 KiB experiment provides a measured simulation baseline for evaluating architectural changes: 801 cycles, or 8.01 microseconds at 100 MHz. The following options are proposed research directions; none has been implemented or measured in this experiment. Published Ascon hardware supports multiple rounds per cycle, including two- and four-round configurations [11]. OpenTitan also identifies multiple rounds per cycle as a way to reduce permutation latency [12]. These sources establish the technique, not a performance result for this FPGA design.

A. Round unrolling and conditional cycle estimates

The current permutation engine performs one round per cycle. With no associated data, the 64 full message blocks require 64 eight-round permutations, in addition to twelve-round initialization and finalization. This accounts for $12 + 64 \times 8 + 12 = 536$ round-computation cycles. The remaining 265 cycles include controller transitions, input/output handshakes, and completion work. They cannot all be assumed removable.

Round unrolling places consecutive rounds in the combinational path between state registers. Two rounds per cycle would reduce the round-computation portion to 268 cycles; four would reduce it to 134. Holding the remaining 265 cycles unchanged gives the estimates below. Each estimate assumes that the larger combinational circuit still meets the 10 ns clock period and that controller behavior is otherwise unchanged.

Architecture at 100 MHz	Total cycles	Latency	Reduction
Current one-round core: measured in XSim	801	8.01 µs	Baseline
Two rounds per cycle: conditional estimate	533	5.33 µs	33.5%
Four rounds per cycle: conditional estimate	399	3.99 µs	50.2%

Conditional architecture estimates for the same 1 KiB message and empty associated data. Only the baseline is a simulation result; optimized timing and resource use remain unverified.

Two rounds per cycle is the first candidate to evaluate because it offers a substantial cycle reduction with a smaller increase in logic depth than four rounds. Unrolling increases logic and routing demands, so a lower cycle count is useful only if routed timing still meets 100 MHz. The estimates are derived from this workload and must not be presented as published benchmark measurements.

B. Other ways to reduce waiting

Option	Expected benefit and practical limit
Overlap output transfer and permutation	A registered output buffer can retain the completed ciphertext while the next permutation runs. The current controller waits for output acceptance before starting that permutation. Benefit depends on stalls; ready/valid behavior must remain correct.
Reduce controller handoff cycles	Inspect the start, wait, and completion transitions for avoidable idle cycles. A cycle trace is needed before assigning a numerical saving.
Multiple cores for independent chunks	Separate authenticated chunks can run concurrently, improving completion time for large files. Each chunk needs a unique nonce and tag, with chunk order and model identity authenticated. Transfer bandwidth can limit the benefit.
Pipeline registers within a round	Can shorten the combinational path, but at a fixed clock may add single-message cycles. Consecutive rounds and message blocks depend on the previous state; independent messages are needed to exploit interleaving.

The existing 128-bit input interface already matches the Ascon-AEAD128 message rate. Widening the external bus alone does not remove the sequential permutation dependency. The standard round counts must remain unchanged; the optimization is to perform the required rounds more efficiently, not to reduce cryptographic work.

C. Proposed comparison with the baseline

The next study should record a baseline cycle trace, implement a two-round permutation variant, and repeat the official-vector tests and the same 1 KiB encryption/decryption experiment. Report cycle latency, routed timing at 100 MHz, LUT and register use, and estimated power for both designs. Simulation, Vivado implementation estimates, and physical-board measurements should be reported separately. This comparison would determine whether the predicted cycle reduction is achieved without compromising functional correctness or timing closure.

XI. Ascon behavior on trained weight bytes

A follow-up experiment on October 6, 2026 tested how the existing Ascon VHDL core responds to changed authenticated inputs, delayed handshakes, and a valid command after authentication failures. All 17 checks across 12 scenarios passed. The input was the same 1,024-byte trained GPT-2 weight sample from Section IX. Five valid scenarios each tested encryption and decryption; seven modified-input scenarios tested rejection during decryption.

The runner verification/gpt2_behavior.py created a separate Vivado 2023.2 XSim project without modifying the RTL or the original testbench. Before generating expected outputs, its Python oracle passed all 1,089 pinned official known-answer vectors. The new testbench compared all 64 payload blocks, encryption tags, authentication events, commit and discard signals, and completion cleanup. Decryption outputs in failed cases were checked against the oracle as provisional plaintext, rather than accepted model weights.

Scenario	Observed behavior	Cycles
Original sample, empty AD	Exact encryption and recovery; authentication passed	801 each
Ciphertext bit flip at byte 0, 512, or 1,023	All three rejected; no commit; discard asserted	801 each
Tag bit flip	Rejected; no commit; discard asserted	801
Wrong key or wrong nonce	Both rejected; no commit; discard asserted	801 each
Valid command after failed commands	Correct encryption and authenticated recovery	801 each
Valid 16-byte associated data	Correct encryption and authenticated recovery	821 each
Associated-data bit flip	Rejected; no commit; discard asserted	821
Input gaps, output stalls, delayed tag	Correct data; held outputs stable; no early authentication	1,003 each
Valid changed nonce	Different ciphertext and tag; correct authenticated recovery	801 each

Table V. XSim behavior results for the 1 KiB trained-weight sample. Each denotes encryption and decryption where both were tested.

The baseline latency remained 801 cycles, or 8.01 microseconds at the simulated 100 MHz clock. Adding 16 bytes of associated data increased it to 821 cycles, or 8.21 microseconds. The stall scenario inserted three-cycle gaps before input beats, held each output beat for three cycles, and delayed tag acceptance by seven cycles. It completed in 1,003 cycles, or 10.03 microseconds. Payload and tag values remained stable while held. These are command-to-done simulation intervals under the specified driver schedule; no physical FPGA or host-transfer time was measured.

Every modified-input case produced auth_ok=0, commit=0, and discard=1. A valid transaction immediately after the failures succeeded. The core still streams provisional plaintext before checking the tag, so an external receiver must quarantine the entire transaction until commit and discard it on failure. This experiment verifies the core signals, not a production authenticated checkpoint loader. Equal valid and invalid latencies in the tested cases do not establish protection against timing, power, or electromagnetic side channels.

XII. GPT-2 behavior after weight recovery

The full GPT-2 model produced identical results when its tested weight slice was replaced with the newly recovered RTL bytes. This experiment used the complete 497,904,640-byte trained FP32 checkpoint, but only the first 1 KiB of parameter data passed through Ascon RTL. That slice contains 256 FP32 values from token embedding row 0; it begins immediately after the checkpoint header. Whole-checkpoint encryption was not performed.

The runner verification/gpt2_model_behavior.py loaded two model instances using the actual checkpoint tensor order and padded vocabulary format. One instance used the original checkpoint bytes; the second substituted the recovered slice. Every model parameter matched. Reserializing both instances reproduced the original full-checkpoint SHA-256, c5acb338337d9f14b814a21e1d18415ce72c34bf18650c9ead5299b4f3ad2733. This check binds the loaded models to the trained export and guards against a wrong tensor order or an unintended model source.

Inference ran in evaluation mode on the NVIDIA RTX 3080 Laptop GPU using PyTorch 2.11.0+cu128. FP32 arithmetic, deterministic operations, and disabled TF32 controlled the numerical comparison. Five text prompts and an explicit token sequence [0, 1, 2, 3] were evaluated. Token 0 directly exercises the embedding row containing the recovered values. Each text prompt also generated 24 new tokens using greedy decoding and a separate seeded top-40 sampling run at temperature 0.8 with seed 20261006.

Comparison	Measured result
Parameters and serialized checkpoint hash	Exactly identical
Prompt logits across six input sequences	1,306,682 FP32 values; all exactly identical
Maximum absolute prompt-logit difference	0.0
Greedy generation	Five prompts; 24 new tokens each; identical token sequences
Seeded generation	Five prompts; 24 new tokens each; identical token sequences
Held-out validation sample	20 batches of 64 predicted tokens; 1,280 tokens total
Mean cross-entropy with original weights	4.0104284048
Mean cross-entropy with recovered slice	4.0104284048
Validation logits and per-batch losses	Exactly identical in all 20 batches

Table VI. Full-model inference comparison with the original and RTL-recovered 1 KiB weight slice.

Validation used the first 20 non-overlapping 64-token input batches from the held-out Tiny Shakespeare token file, with next-token targets. Its mean cross-entropy was 4.0104284048 for both model instances. Section VIII reports a different five-batch evaluation mean of 3.810759 after training; the different sampling schedules prevent a direct before-and-after quality comparison between those two means. The behavior test compares original and restored weights on the same inputs and obtains zero change.

Together, these results demonstrate exact recovery of the tested weight bytes, rejection of the seven tested modifications, and preservation of full-model inference behavior after restoring that slice. They do not establish model quality, whole-checkpoint RTL encryption, or a physical-board application. The public evidence folder paper/evidence/gpt2_behavior_20261006 preserves the runner and testbench snapshots, compact simulator results, and machine-readable report.json and model_report.json, including generated text and token sequences. Trained weight binaries, full block dumps, and generated Vivado projects remain local.

XIII. Limitations and next work

This version is an unprotected cryptographic core. It does not prevent power or electromagnetic side-channel analysis, and it does not detect injected faults. Kandi et al. describe threshold and redundancy-based hardware options that could be studied after the baseline is stable [3]. A faster unrolled or ping-pong architecture is another option, but it would use more logic and make the first implementation harder to inspect [4]. The external quarantine buffer is also a required part of safe decryption, not an optional application feature.

XIV. Conclusion

The project now has a readable NIST Ascon-AEAD128 RTL core, an explicit streaming protocol, an authentication-safe decryption contract, official-vector tests, and a completed Vivado synthesis and implementation flow. GHDL and XSim each passed all 1,089 official vectors in encryption and decryption. The routed design met the 100 MHz constraint, and Vivado generated a bitstream. The remaining work is to program the Nexys A7 with the corrected configuration properties and verify the four-vector demonstration on physical hardware. The GPT-2 application experiment also demonstrated the complete path from GPU fine-tuning to exported weight bytes and authenticated VHDL encryption. A 1 KiB sample passed encryption and decryption in 801 cycles per direction at the testbench clock. Follow-up behavior tests passed all 17 checks, including tamper rejection and stalled transfers. Restoring the recovered slice into the full GPT-2 model preserved parameters, logits, generated tokens, and validation losses exactly. Whole-checkpoint GPT-2 encryption, physical transfer overhead, and board-level measurements remain future work.

References

[1] M. S. Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, “Ascon-Based Lightweight Cryptography Standards for Constrained Devices,” NIST SP 800-232, Aug. 2025, doi: 10.6028/NIST.SP.800-232.

[2] Ascon Team, “ascon-c: Reference and optimized implementations of Ascon,” commit 446347f21b209f3921c65ece70027c366cbe1693. [Online]. Available: https://github.com/ascon/ascon-c

[3] A. Kandi et al., “Hardware Implementation of ASCON,” NIST Lightweight Cryptography Workshop, June 2023.

[4] K. Liu, L. Han, and Z. Quan, “A High Performance Ascon Architecture for Data Encryption and Authentication,” in Proc. 5th Int. Conf. on Cryptography, Network Security and Communication Technology (CNSCT 2026), Haikou, China, Jan. 23–25, 2026, doi: 10.1145/3802927.3802941.

[5] N. T. M. Huong, T. V. Nam, and P. V. Phat, “Application of the Ascon-AEAD128 Algorithm for IoT Data Security,” 11th Int. Conf. on Applying New Technology in Green Buildings, 2026, doi: 10.1109/ATIGB70203.2026.11628316.

[6] Digilent, “Nexys A7-100T Master XDC.” [Online]. Available: https://github.com/Digilent/digilent-xdc

[7] Microsoft, “Install WSL.” [Online]. Available: https://learn.microsoft.com/en-us/windows/wsl/install. Accessed Oct. 6, 2026.

[8] NVIDIA, “CUDA on WSL User Guide.” [Online]. Available: https://docs.nvidia.com/cuda/wsl-user-guide/. Accessed Oct. 6, 2026.

[9] SPIRE-GMU, “SAGE_Accelerator, llmc-gpt2,” commit 4e586d80b61ad7577c64391202a8c43104728f1d. [Online]. Available: https://github.com/SPIRE-GMU/SAGE_Accelerator/tree/4e586d80b61ad7577c64391202a8c43104728f1d/llmc-gpt2.

[10] PyTorch, “Start Locally.” [Online]. Available: https://pytorch.org/get-started/locally/. Accessed Oct. 6, 2026.

[11] R. Primas, “Hardware Design of Ascon (SP800-232),” ascon-verilog. [Online]. Available: https://github.com/rprimas/ascon-verilog. Accessed Oct. 6, 2026.

[12] OpenTitan, “Ascon: Theory of Operation.” [Online]. Available: https://opentitan.org/book/hw/ip/ascon/doc/theory_of_operation.html. Accessed Oct. 6, 2026.

Appendix A Reproducing the environment and training

Commands are grouped by the shell in which they run. WSL installation uses administrator PowerShell; the project and training commands run in Ubuntu. The following commands document the completed setup. Package versions observed in Table II should be retained when reproducing the numerical results, because an unpinned requirements installation may select newer releases.

A1 Windows PowerShell setup

```
wsl --install -d Ubuntu-22.04
# Restart Windows if requested, then continue:
wsl --update
wsl --list --verbose
wsl -d Ubuntu-22.04
```

Create the Ubuntu username and password on first launch. Confirm VERSION 2 in the distribution list. Password characters are intentionally invisible at the prompt. WSL GPU access was then checked inside Ubuntu with nvidia-smi.

A2 Ubuntu project and Python environment

```
cd ~
sudo apt update
sudo apt install -y git build-essential python3-pip python3-venv
mkdir -p ~/projects
cd ~/projects
git clone https://github.com/SPIRE-GMU/SAGE_Accelerator.git
cd SAGE_Accelerator
git checkout 4e586d80b61ad7577c64391202a8c43104728f1d
cd llmc-gpt2
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install torch --index-url https://download.pytorch.org/whl/cu128
python -m pip install -r requirements.txt
nvidia-smi
```

The checkout command above pins the source revision used in this experiment. For later sessions, return to llmc-gpt2 and reactivate .venv. No Linux NVIDIA display driver or separate CUDA compiler installation was needed for the PyTorch workflow [8], [10].

A3 CUDA calculation check

```
python3 - <<'PY'
import torch
print(torch.__version__)
assert torch.cuda.is_available()
print(torch.cuda.get_device_name(0))
x = torch.randn(256, 256, device="cuda")
y = x @ x
torch.cuda.synchronize()
print(y.shape)
PY
```

The closing PY must be on a line by itself. During setup, visible bracketed-paste markers were resolved by typing bind 'set enable-bracketed-paste off' in Bash. This changes terminal input behavior and has no effect on model results.

Appendix A Dataset and correctness checks

```
python3 dev/data/tinyshakespeare.py
python3 train_gpt2.py --device cuda --model gpt2 \
  --num_iterations 0 --write_tensors 1 --batch_size 4 \
  --sequence_length 64 --total_batch_size 256
make test_gpt2
OMP_NUM_THREADS=8 ./test_gpt2
```

Require overall okay: 1 from the C test. Use batch size 4 for this reference because the ten expected losses in test_gpt2.c are fixed to that workload. A zero-update reference run reports nanms because it has no training durations to average.

A4 Final weight export added after the training loop

Append the following block at the end of train_gpt2.py, inside its main block. Its first line has four leading spaces. It saves updated FP32 weights and selects actual weight bytes after the 1,024-byte model header.

```
    if master_process and args.num_iterations > 0:
        export_dir = args.output_dir or "."
        os.makedirs(export_dir, exist_ok=True)
        export_path = os.path.join(export_dir, "gpt2_trained_fp32.bin")
        write_model(raw_model, export_path, dtype="float32")
        file_bytes = os.path.getsize(export_path)
        print0(f"Trained checkpoint: {file_bytes} bytes")
        print0(f"Weight payload including vocabulary padding: {file_bytes - 1024} bytes")
        with open(export_path, "rb") as source:
            source.seek(1024)
            sample = source.read(1024)
        assert len(sample) == 1024
        with open(os.path.join(export_dir, "weights_1KiB.bin"), "wb") as target:
            target.write(sample)
        print0("Exported 1024 weight bytes for Ascon")
```

```
python3 -m py_compile train_gpt2.py
```

The export is guarded by a positive iteration count so a reference-only run cannot be mislabeled as trained weights. The original before-training bridge files remain distinct from the final trained checkpoint.

Appendix A Training transfer and XSim execution

A5 Ubuntu GPU fine-tuning command

```
python3 train_gpt2.py \
  --device cuda --model gpt2 \
  --input_bin dev/data/tinyshakespeare/tiny_shakespeare_train.bin \
  --input_val_bin dev/data/tinyshakespeare/tiny_shakespeare_val.bin \
  --batch_size 1 --sequence_length 64 --total_batch_size 256 \
  --num_iterations 40 --overfit_single_batch 0 \
  --val_loss_every 10 --val_max_steps 5 --write_tensors 0 \
  --output_dir runs/ascon_demo_40
```

A6 Inspect and transfer the exported sample

```
stat -c "%n %s bytes" runs/ascon_demo_40/gpt2_trained_fp32.bin
stat -c "%n %s bytes" runs/ascon_demo_40/weights_1KiB.bin
sha256sum runs/ascon_demo_40/weights_1KiB.bin
cp -i runs/ascon_demo_40/weights_1KiB.bin \
  /mnt/c/Users/Symok/Downloads/gpt2_trained_weights_1KiB.bin
```

A7 Windows PowerShell VHDL simulation

```
cd "C:\Users\Symok\Downloads\ascon-aead128-vhdl-main"
python verification/gpt2_weights_demo.py `
  "C:\Users\Symok\Downloads\gpt2_trained_weights_1KiB.bin"
```

The runner uses Vivado on PATH or the installed C:\Xilinx\Vivado\2023.2\bin\vivado.bat. It first validates the Python oracle and then invokes XSim using a generated testbench. The project and results are isolated under build/gpt2_weights_demo. Before rerunning, archive that output directory if the previous receipts must be retained.

Artifact	Purpose
report.json and inputs.json	Inputs, sample hash, timing definition, and verified result
simulation_results.txt	VHDL ENCRYPT and DECRYPT PASS lines and 801-cycle counts
full_output.txt	All 64 output blocks in each direction
ciphertext.bin and tag.bin	1,024-byte ciphertext and 16-byte authentication tag
recovered_weights.bin	Byte-exact decryption result
vivado_console.log	Compile, elaboration, and simulation receipts
runs/ascon_demo_40/main.log	Ubuntu training and validation losses

The training output, source revision, installed software versions, sample hash, and simulation receipts jointly establish the provenance of this experiment. Reported training memory is PyTorch allocation; reported encryption time is simulation latency. Neither is a physical board measurement.